

# Relatório de Segurança

Proteções XSS e CSRF — Contexto, Justificação e Metodologia

Sistema de Loja de Informática · 2026

## 1. O que são XSS e CSRF

O Cross-Site Scripting, habitualmente abreviado como XSS, é um tipo de ataque que consiste em injetar código JavaScript malicioso numa página web. Quando um utilizador visita essa página, o código executa no seu browser, podendo roubar cookies de sessão, redirecionar para sites fraudulentos ou capturar credenciais introduzidas nos formulários. O ataque tira partido da confiança que o browser deposita no conteúdo que recebe do servidor.

O Cross-Site Request Forgery, conhecido como CSRF, funciona de forma diferente. Aqui o atacante engana um utilizador autenticado a enviar, sem saber, um pedido HTTP para uma aplicação onde já tem sessão ativa. Por exemplo, um utilizador com sessão aberta num banco online visita um site malicioso que, em segundo plano, submete um formulário invisível para transferir dinheiro. O servidor, ao receber o pedido com os cookies legítimos do utilizador, não consegue distinguir se a ação foi intencional ou forçada.

## 2. Por que não foram implementadas neste projeto

A razão é simples e direta: este projeto é uma aplicação desktop desenvolvida em Python com a biblioteca Tkinter. Não existe nenhum browser envolvido, não há pedidos HTTP entre cliente e servidor web, e não existe renderização de HTML. A interface é construída diretamente pelo sistema operativo, através de widgets nativos, o que elimina por completo o contexto em que XSS e CSRF operam.

*XSS e CSRF são ameaças exclusivas de aplicações web que correm no browser. Uma aplicação desktop como esta não tem superfície de ataque para nenhuma das duas vulnerabilidades. Implementar proteções contra elas seria tecnicamente incorreto e induziria em erro quem analisasse o código.*

A comunicação desta aplicação faz-se diretamente com a base de dados MariaDB através de um driver Python, sem qualquer intermediário HTTP. Não existem formulários HTML, não existem cookies de sessão web, e não existe um servidor a servir páginas. Por este motivo, as proteções implementadas foram as que realmente fazem sentido neste contexto, nomeadamente a validação de inputs, o uso de prepared statements contra injeção SQL, o hashing de passwords com bcrypt, e a limitação de tentativas de login.

### 3. Como se protegeria contra XSS se fosse uma aplicação web

Numa aplicação web, a primeira linha de defesa contra XSS consiste em nunca colocar diretamente no HTML qualquer dado que venha do utilizador. Todos os valores devem passar por um processo de escaping antes de serem renderizados, convertendo caracteres especiais como o sinal de menor, o sinal de maior e as aspas nas suas entidades HTML equivalentes, tornando-os inofensivos para o browser.

Num projeto Flask, por exemplo, o motor de templates Jinja2 faz este escaping automaticamente quando se usa a sintaxe de duplas chavetas. Para situações em que seja necessário renderizar HTML de forma explícita, existe a função Markup que sinaliza o conteúdo como seguro, devendo ser usada apenas com dados previamente sanitizados.

```
# Flask com Jinja2 – escaping automático
# O template renderiza assim:
# <p>{{ nome_utilizador }}</p>
#
# Se nome_utilizador = '<script>alert(1)</script>'
# O browser recebe: &lt;script&gt;alert(1)&lt;/script&gt;
# e trata-o como texto – não como código.
```

Uma segunda camada de proteção é a Content Security Policy, uma diretiva enviada no cabeçalho HTTP que instrui o browser a só executar scripts provenientes de origens explicitamente autorizadas, bloqueando qualquer script injetado pelo atacante, mesmo que chegue a ser inserido na página.

```
# Cabeçalho HTTP Content-Security-Policy
Content-Security-Policy: default-src 'self'; script-src 'self';
object-src 'none'
#
# Em Flask:
response.headers['Content-Security-Policy'] = "default-src 'self'"
```

### 4. Como se protegeria contra CSRF se fosse uma aplicação web

A proteção contra CSRF assenta num mecanismo de tokens. O servidor gera um valor aleatório e imprevisível, associa-o à sessão do utilizador, e inclui-o como campo oculto em todos os formulários. Quando o formulário é submetido, o servidor valida que o token recebido corresponde ao que emitiu. Como um site malicioso não tem acesso ao token, os seus pedidos forjados são automaticamente rejeitados.

```
# Flask-WTF gere os tokens CSRF automaticamente
from flask_wtf import FlaskForm
from wtforms import StringField

class LoginForm(FlaskForm):
    email = StringField('Email')
    password = StringField('Password')

# No template HTML:
# <form method='POST'>
# {{ form.hidden_tag() }} <!-- insere o token CSRF -->
# ...
# </form>
```

Uma medida complementar é configurar os cookies de sessão com o atributo SameSite definido como Strict ou Lax. Este atributo instrui o browser a não enviar o cookie em pedidos originados noutros domínios, cortando o vetor principal que o CSRF utiliza para se concretizar.

```
# Configuração dos cookies de sessão em Flask
app.config['SESSION_COOKIE_SAMESITE'] = 'Strict'
app.config['SESSION_COOKIE_SECURE'] = True # apenas sobre HTTPS
app.config['SESSION_COOKIE_HTTPONLY'] = True # inacessível ao JavaScript
```

## 5. Conclusão

A segurança de um sistema não se mede pelo número de proteções implementadas, mas pela adequação dessas proteções ao contexto real da aplicação. Implementar tokens CSRF ou políticas de Content Security Policy numa aplicação desktop seria o equivalente a colocar um cadeado numa porta que não existe.

Este projeto implementou as proteções que fazem sentido para uma aplicação desktop com acesso direto a base de dados: validação e sanitização de todos os inputs do utilizador, proteção contra injeção SQL através de prepared statements, armazenamento de passwords com hash bcrypt, controlo de tentativas de login com bloqueio temporário, gestão de sessão com expiração por inatividade, e logging de eventos de segurança para monitorização.

Caso este projeto evoluísse para uma plataforma web, as proteções XSS e CSRF descritas neste documento seriam obrigatórias e deveriam ser implementadas desde o primeiro dia de desenvolvimento, preferencialmente através de bibliotecas maduras como Flask-WTF, que automatizam grande parte da complexidade envolvida.